A FIELD GUIDE · FROM THE LOW DESERT MOTEL

LOW DESERT ENGINE

How a first-person world is built in three.js, with no build step and no assets, and how to build a completely different one on the same engine.



COVERS
Light, shadow and texture · walking, eye level and sitting · doors · the day/night cycle · cars · building new worlds

BUILT FROM
The game as of Round 48: 29 scripts, about 24,000 lines, three.js r128

EDITION
October 2026

CONTENTS

WHAT IS IN HERE

The Low Desert Motel started as one HTML file and grew into a Route 66 motel, a desert with seven landmarks, a diner, a speakeasy forty feet under the ground, an auditorium, an ice rink and a 1950s living room seen through observation glass. Almost none of the code behind those places is about motels. It is a small, general engine for walkable first-person worlds. This guide pulls that engine apart so it can be handed to someone, or to another AI, and used to build something else entirely.

1 **THE ENGINE AT A GLANCE**
What it is, how the files fit together, and the numbers everything else depends on.

2 **GRAPHICAL STYLE: LIGHT, SHADOW, TEXTURE**
The painted-colour look, the six layers of light, shadows that touch their casters, canvas textures, signs, and how a whole world becomes a few hundred draw calls.

3 **WALKING, RUNNING, SPRINTING, SITTING**
Three gaits, eye level that follows the floor, collision that never traps you, and how sitting and lying down work.

4 **DOORS AND DOORWAYS**
The door leaf, the hole it hangs in, and the rules that come from forty-six doors that each failed at least once.

5 **THE DAY/NIGHT CYCLE**
One number drives the sun, the sky, the fog, every lamp and every lit sign.

6 **CARS**
How the cars are modelled, the rigid-body and tyre physics, the chase camera, headlights, and the highway traffic.

7 **BUILDING A DIFFERENT WORLD**
Coordinates, local frames, rooms, terrain, underground spaces, performance, atmosphere, and the checks that keep a world from breaking.

A **REFERENCE CARD**
Every registration function, the controls, and a porting checklist on two pages.

HOW TO READ THE NUMBERS

Units are metres, seconds and radians. `y` is up. At yaw 0 the camera looks down `-z`, which the compass calls north, so `+x` is east and `+z` is south. Every number quoted here is the one in the game's source, not an idealised version of it. Where the source and good practice disagree, the guide says so.

CHAPTER ONE

1 THE ENGINE AT A GLANCE

A hand-rolled first-person engine on three.js r128. It has no bundler, no modules, no build step and no asset files: open `index.html` by double-clicking it and the whole world is generated in the browser. Every texture is drawn on a canvas at load time, every static object is merged into a handful of meshes, and the simulation runs at a fixed 60 steps a second.

EYE HEIGHT
1.66 m
above whatever you stand on

BODY
r 0.34 m
tested from 0.12 to 1.72 m above the feet

GAITS
3.05 · 5.6 · 10.4
walk · run (Shift) · sprint (Q), m/s

STEP-UP
0.62 m
highest ledge you climb without a ramp

SIMULATION
1/60 s
fixed step, at most 6 per frame, frame time capped at 0.1 s

LENS
88° across
vertical held 50–66°; near 0.24 m, far 4,000 m

A FULL DAY
414 s
about 7 minutes for 24 game hours

REAL LIGHTS
1 + 12
one shadow-casting sun or moon, twelve pooled point lights

HOW IT IS PUT TOGETHER

The page loads 29 classic `<script>` tags in order. They share one global scope, so a `const` declared in an early file is visible to every later one, exactly as if it were one long file. The numbering on the file names *is* the load order, and nothing reorders itself.

FILE	ROLE	REUSABLE AS-IS?
`00-config.js`	Spawn point, eye height, player radius, site constants, image slots	Yes, after you edit the constants
`01-helpers.js`	Seeded random numbers, geometry makers, merge buckets, `push()`, the bake, GIF decoder	Yes
`02-textures.js`	Every canvas texture, plus the sign helpers `signTex`, `signPanel`, `fitText`	Keep the helpers, replace the textures
`03-scene.js`	Renderer, tone-mapping shoulder, camera and lens, base lights, flashlight, the sun's shadow camera	Yes
`04-terrain.js`	Analytic height field, level-of-detail rings, graded pads, `groundAt`	Keep the machinery, replace the landscape
`05-sky.js`	Sky dome, stars, sun and moon, clouds, birds, highway traffic, the `DayNight` clock	Yes; repaint the palette
`06-collision.js`	Colliders, walking surfaces, voids, zones, buried volumes, seats, the collider grid	Yes

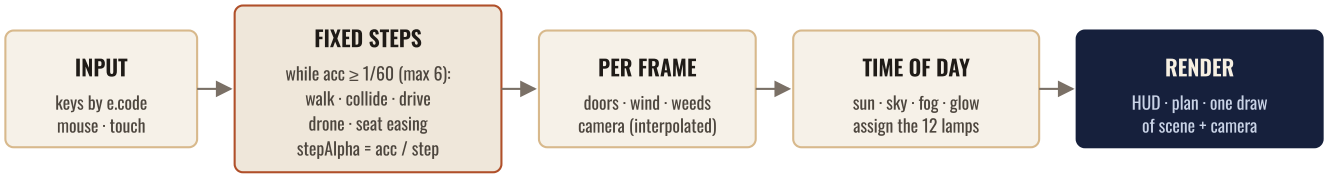
`07` to `14j`, except `11`	The content: site, motel, pool, office, signs, canteen, desert, diner, speakeasy, auditorium, warehouse, baths, rink, observation room	No. This is what you replace
`11-driving.js`	Car rigid body, four springs, tyre model, headlights	Yes
`15-bake.js`	Materials for the glowing buckets, then merge everything and build the collider grid	Yes; edit the bucket list
`16-lighting.js`	The pool of twelve point lights that follows the camera	Yes
`17-player.js`	Input, movement, collision response, doors, seats, the drone	Yes
`18-hud.js`	Compass, clock, location name, toasts, the site plan	Mostly
`19-loop.js`	Fixed-timestep loop, cameras, day/night application, ambience, the `MOTEL` debug handle	Yes, minus the motel-specific bits

RULE · ORDER IS LOAD-BEARING

A function declaration only hoists within its own script. If a file's *top-level* code calls a function declared in a later file, the page throws on load. Calling a later file's function from *inside* a function body is fine, because by the time anything is called every script has run. That is why the sign helpers live in `02-textures.js`: half the world raises a sign before `12-signage.js` loads. Any registry that several files push into (`GLOW`, `ANIM`, `LAMPS`, `DOORS`) must be declared before all of them.

ONE FRAME

The simulation and the picture run on separate clocks. Physics, walking and driving advance in fixed 1/60 s steps from an accumulator, so the game behaves the same at 30 fps as at 144. Anything drawn from simulated state (the car, the drone) is drawn *between* its last two steps using `stepAlpha`, so motion stays smooth whatever the display rate.



requestAnimationFrame → frame(): delta = min(now − last, 0.1); acc += delta

```
function frame(){
  requestAnimationFrame(frame);
  let delta = Math.min(now - last, 0.1);        // a stalled tab cannot fling you through a wall
  acc += delta; let guard = 0;
  while(acc ⩾ STEP && guard++ < 6){             // STEP = 1/60
    if(driving) driving.veh.savePrev();         // one step of history, for drawing between steps
    fixedUpdate(STEP);
    acc -= STEP;
  }
  stepAlpha = clamp(acc/STEP, 0, 1);
  // ...camera, day/night, lamps, emissives, HUD...
  renderer.render(scene, camera);
}
```

FIVE IDEAS THE WHOLE ENGINE RESTS ON

01 **Paint, don't light, the colour.** Every surface carries a hex you chose, stored as a vertex colour. The lighting shades it but never decides it, and a tone-mapping shoulder stops bright paint clipping to white.

02 **Few real lights, many fake ones.** One sun, twelve point lights reassigned every frame to the lamps nearest the camera, and emissive or unlit materials for everything else that glows.

03 **Merge everything that does not move.** Geometry is pushed into named *buckets* by material, and each bucket bakes into one mesh. A world of tens of thousands of pieces draws in a few hundred calls.

04 **Collision is boxes and floors, not meshes.** Axis-aligned boxes with a vertical span block you, and a separate list of flat plates and ramps says what you stand on. Geometry and collision are authored side by side and must agree.

05 **Everything is generated.** Textures are drawn on canvases, terrain is a function, and furniture is primitives. The page needs nothing but three.js. Real pictures are optional overrides.

CHAPTER TWO

2 GRAPHICAL STYLE

The look in one sentence: painted, literal colour under soft desert light. Every surface is a hex someone chose, shaded by a warm sky, one hard sun and a handful of lamps, with the brightest highlights rolled off rather than clipped. It is closer to a diorama than to photorealism, and it holds together because nothing is allowed to go flat white or flat black.



Golden hour on the motel court. A single shadow-casting sun low in the west, the warm hemisphere fill, the painted stucco and teal trim. No texture in this frame is a photograph.

THE RENDERER

SETTING	VALUE	WHY
Antialiasing, pixel ratio	`on · min(dpr, 2)`	Crisp edges without paying for 3× screens
Output colour space	`sRGB`	Canvas textures are flagged sRGB too, so painted hexes arrive as painted
Shadow map	`PCFSoftShadowMap`	Soft edges at one shadow caster's cost
Tone mapping	`Custom shoulder at 0.76`	Below 0.76 the output equals the input; above it each channel rolls off towards 1.0
Anisotropic filtering	`min(8, GPU max)`	Floors and roads stay sharp at a grazing angle
Near / far planes	`0.24 m / 4,000 m`	Depth precision is set by the near plane. At 0.08 distant signs buzzed; 0.24 gives about 6 cm of resolution 500 m out

The tone-mapping shoulder is the single most important line for the look. Painted colours stay exactly as painted, because that was the reason for avoiding filmic curves. But a white lounger in full sun leaves the shader at 1.8, and without the shoulder every face of it clips to the same 255,255,255 and loses its form. The shoulder works per channel, so a warm highlight stays warm instead of being pulled towards grey.

```
T.ShaderChunk.tonemapping_pars_fragment = T.ShaderChunk.tonemapping_pars_fragment.replace(
  "vec3 CustomToneMapping( vec3 color ) { return color; }",
  `vec3 CustomToneMapping( vec3 color ) {
     color *= toneMappingExposure;
     const float K = 0.76;                              // where the shoulder starts
     vec3 lo = min( color, vec3( K ) );
     vec3 hi = max( color - vec3( K ), vec3( 0.0 ) );
     return lo + ( 1.0 - K ) * ( vec3( 1.0 ) - exp( -hi / ( 1.0 - K ) ) );
   }`);
renderer.toneMapping = T.CustomToneMapping;
```

SIX LAYERS OF LIGHT

Real-time lights are expensive and three.js recompiles shaders when their count changes, so the engine keeps the number of real lights fixed and builds everything else out of materials that glow.

#	LAYER	WHAT IT IS	USED FOR
1	Base fill	`HemisphereLight` sky `#ffe7c4`, ground `#4a2a18`, 0.72 by day; `AmbientLight` `#fff0dc` 0.15 to 0.21	The sky and the warm ground bouncing light into every shadow, so nothing goes black
2	The key	One `DirectionalLight`: the sun by day (`#ffeed0`, 1.12), the moon by night (`#8ea6d4`, about 0.38)	The only shadow caster in the world
3	Lamp pool	Twelve `PointLight`s, no shadows, moved every frame onto the twelve registered lamps nearest the camera (within 46 m)	Every fixture that has to actually light a room: sconces, porch lights, strip lights, desk lamps
4	Glowing surfaces	`MeshStandardMaterial` with emissive, registered in `GLOW` and ramped by the clock	Lampshades, ceiling fittings, vending fronts, lit windows, bulkheads
5	Unlit faces	`MeshBasicMaterial` with `toneMapped:false`; neon panels run 0.40 + 0.60 × dark	Neon and lit signs, headlamp lenses, TV screens, fairy lights
6	Faked light	Additive sprites and transparent discs	Candle pools on tables, the sun and moon discs, lamp halos, the pool's night glow

On top of those sits the flashlight (**L**): a `SpotLight` moved onto the camera every frame (intensity 2.45, 34 m throw, cone 0.40 rad, penumbra 0.55, no shadow) plus a weak 6 m point light half a metre ahead, so the floor at your feet is lit too.

REGISTERING A LAMP

```
LAMPS.push({ x, y, z,                  // world position of the bulb
  color: 0xffd8a0, intensity: 0.42,     // colour and strength before the clock multiplier
  dist: 8, decay: 1.5,                  // range in metres; 1.5 is gentler than physical, so no hot spots
  indoor: true,                         // burns at max(dark, 0.70) all day
  vol: [x0,x1, z0,z1, y0,y1],           // only on while the CAMERA is inside this box
  flicker: false, mothy: false });      // a bad ballast; insects circle it at night
```

Every frame `updateLights()` collects the lamps within 46 m whose volume contains the camera, sorts them by distance and hands the nearest twelve to the pool. The clock multiplier is `dark` for outdoor lamps, `max(dark, 0.70)` for indoor ones, and 1 for anything inside an underground volume.



A motel room at night. The bedside and bathroom fixtures are pool lamps with a `vol` the size of the room, so they never light the walkway through the wall.



The Dry Well, forty feet down. The sun and sky are faded out; fourteen fixtures and candle discs do all the work.

RULES FOR LIGHT

- **An emissive does not illuminate.** A glowing candle flame lights nothing. If a thing must light a room, register a lamp; if the twelve are spoken for, fake the pool of light with an additive disc on the table under it.
- **A lamp is a lens with a dark body.** `emitMat()` gives a glowing surface a near-black diffuse colour, so the point light a few centimetres away cannot add its own light on top of the emissive and push it past white.
- **A lampshade is a tube, and a ceiling fitting is a bowl.** A closed cylinder puts a flat glowing disc across the mouth of the shade, and that becomes the brightest flat area in the room. Use an open, double-sided tube with a bulb inside, and a half-dome under a ceiling rose.
- **A lamp with a volume is on only while you are inside it.** That is what stops light leaking through walls. For a room you look *into* but never enter, such as the living room behind the observation glass, widen the volume to include where the watcher stands.
- **Glow goes through a soft knee.** Emissive intensity is linear up to 0.55 of the brightest channel and then eases towards 0.92, so an amber lamp stays amber however hard it is driven.

SHADOWS THAT TOUCH WHAT CASTS THEM

Only the key light casts shadows. Its shadow camera is an orthographic box 92 m across that follows the player, snapped to whole shadow-map texels so the shadows do not crawl as you walk. The light sits 110 m out along the sun direction from that target, so the depth range is only ±100 m, which is what lets the bias be tiny.

```
key.shadow.mapSize.set(3072, 3072);            // 3 cm a texel across 92 m
const ksc = key.shadow.camera, KEY_D = 110;
ksc.left = -46; ksc.right = 46; ksc.top = 46; ksc.bottom = -46;
ksc.near = KEY_D - 100; ksc.far = KEY_D + 100;
ksc.updateProjectionMatrix();                  // without this, none of the above takes effect
key.shadow.bias = -0.00005; key.shadow.normalBias = 0.012;
const SHADOW_SNAP = (ksc.right - ksc.left) / key.shadow.mapSize.x * 2;

// every frame: follow the player in whole texels, then stand KEY_D out along the sun
const kx = Math.round(camera.position.x / SHADOW_SNAP) * SHADOW_SNAP;
const kz = Math.round(camera.position.z / SHADOW_SNAP) * SHADOW_SNAP;
key.position.set(kx + sunDir.x*KEY_D, kty + sunDir.y*KEY_D, kz + sunDir.z*KEY_D);
key.target.position.set(kx, kty, kz);
key.castShadow = dn.sunHeight > 0.04;          // the moon is too weak to shadow anything but acne
```

RULES FOR SHADOWS

- **Tell the shadow camera.** An `OrthographicCamera` builds its projection once. Change its bounds without calling `updateProjectionMatrix()` and the world is shadowed through the stock 10 m box. That is why the motel once cast no shadow on its own car park.
- **Flat things do not cast.** When the merge bakes a bucket it splits off every piece thinner than 12 mm (decals, rugs, stains, the lot surface) into a second mesh that receives but does not cast. A plane in the depth map sits exactly on the surface it shades and makes acne.
- **See-through things do not cast.** Transparent, additive and depth-less materials are excluded, or every window pane and light shaft prints a solid shadow.
- **Do not raise `normalBias` to cure acne.** A big normal bias lifts the shadow off the foot of whatever casts it, so every post stands on a strip of daylight. Find what is shadowing itself instead. Measured result: the gap between a post and its shadow went from 11 cm to under 2 cm.

UNDER THE GROUND THERE IS NO SKY

Indoors there is still a sky outside the window, so the hemisphere light stays on. Underground there is not. `addBuried(x0,x1, z0,z1, y0,y1, amb)` registers a volume; while the camera is inside one, the loop fades over about a third of a second to:

- sun intensity × 0 (taken away outright);
- hemisphere intensity × 0.18, its colours pulled to lamplight (sky `#ffd2a0`, ground `#3a2418`);
- ambient raised to 0.44 in the volume's own colour `amb`: warm `#ffdcb4` for a lamplit bar, grey-green `#d8dcd4` for a tiled corridor, near-dark `#585c5c` for an observation room.

Lamps inside a buried volume ignore the clock and burn at full strength, and so do emissive materials of the `"buried"` kind. Walking down a stair becomes a transition rather than a cut.



The ice arena. Cold ambient, high-bay fixtures from the pool, grey-blue ice kept below white so it reads as a surface rather than a hole.



The 1955 living room. Its lamps belong to a volume that includes the observation room, so the set stays lit for whoever is watching.

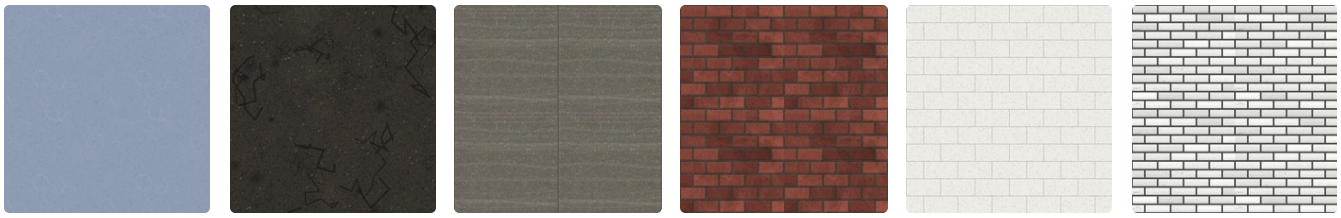
ATMOSPHERE

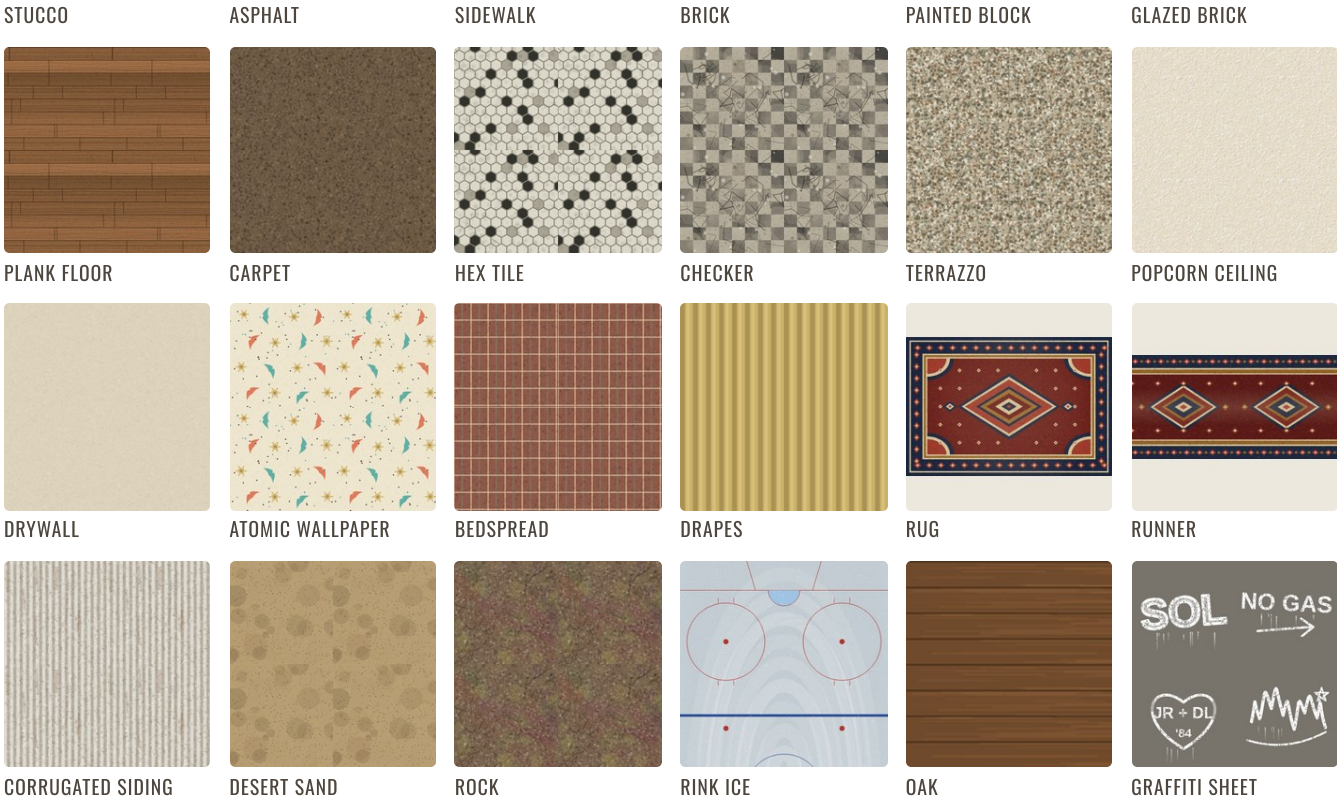
Linear fog does the work of distance. By day its colour equals the sky's horizon colour, so a far mesa melts into heat haze. At night the haze is pulled towards `#0a1020` and made *darker* than the sky, or the mountains wash out into it. Fog starts at 185 m and is full at 1,460 m by day (237 m and 1,790 m at night), and a dust gust tints it and pulls it in by up to 46%. The sky itself is a 2,400 m sphere drawn from a 16 × 256 canvas gradient that is repainted every frame (chapter 5). Three cloud layers (cumulus, altocumulus, cirrus) are sprites drawn on canvases, tinted by the sky each frame, and drifting downwind.

TEXTURES: DRAWN, NOT LOADED

Every map in the world is drawn into a `<canvas>` at load time, almost always in three passes: a **base fill**, thousands of tiny **noise dots** in two or three tints, then **strokes and features** (joints, grain, cracks, trowel swirls). Sizes run from 64 to 512 px. The result is flagged sRGB, set to repeat, and given anisotropic filtering.

```
TEX.stucco = (function(){                          // slate-periwinkle motel walls
  const c = cvs(256,256), x = c.getContext("2d");
  x.fillStyle = "#8e9cb4"; x.fillRect(0,0,256,256);                    // 1 · base fill
  noiseDots(x,256,256,2600,["rgba(255,255,255,0.06)","rgba(0,0,0,0.06)",
                            "rgba(120,140,170,0.10)"]);                 // 2 · noise
  for(let i=0;i<70;i++){                                               // 3 · trowel swirls
    x.strokeStyle = "rgba(255,255,255,"+(0.02+Math.random()*0.05)+")";
    x.lineWidth = 1+Math.random()*3; x.beginPath();
    x.arc(Math.random()*256, Math.random()*256, 6+Math.random()*22, Math.random()*6, Math.random()*6);
    x.stroke();
  }
  return rep(setSRGB(new T.CanvasTexture(c)), 1, 1);
})();
```





WORLD-SCALED UVS

`boxGeo(w, h, d, uvScale)` rescales each face's 0–1 UVs by that face's real size, so one texture tiles at the same density on a kerb and on a wall. `uvScale` is repeats per metre (0.35 by default; brick around 0.8; fine fabric 2 or more). This is what lets thousands of differently sized boxes share one material and merge into one mesh. `planeGeo(w, h, uvScale)` does the same for planes.

COLOUR COMES FROM THE VERTEX, AND CAN ONLY DARKEN

Bucket materials are white, or carry a neutral map. Each piece pushed into a bucket carries its own hex, which the merge writes into a vertex-colour attribute. A vertex colour multiplies the map, so **it can only ever darken the texture under it.** Pick the bucket for the colour you want to end at, not for the material you would name the thing after. Twenty-four "spalled concrete" patches pushed into the gravel bucket came out orange-red, because the gravel map is `#8a5f45`.

DECALS: A DIFFERENT CROP EVERY TIME

- `streakGeo(w,h)` and `cellGeo(w,h,cols,rows)` pick a random cell of a 2×2 decal sheet, at a random offset, scale and flip. Each sheet fades to nothing at its margins, so a crop never lands on a hard edge and forty stains never repeat the same shape.
- `tagGeo(w,h,ci,ri)` names its cell, for graffiti where the pieces are not interchangeable.
- `puddleGeo(rx,rz)` is an irregular disc lying in xz. A rectangle of dark water reads as a slab someone laid on the floor.
- `runnerGeo(L,W)` keeps a rug's border on its long edges and its fringe on its ends, which is how rugs are made.

Stains and age that must sit *in* a surface use a lit material with a vertex colour below the surface's own. An unlit stain is a constant brightness, so in the dim half of a room it ends up brighter than the plaster and reads as fog on the wall.

SIGNS AND PRINTED FACES

```
const tex = signTex(Math.round(w*240), Math.round(h*240), (x, W, H) ⇒ {   // ~240 px a metre
  x.fillStyle = "#24365a"; x.fillRect(0, 0, W, H);
  x.fillStyle = "#e8e6dc"; fitText(x, "STAFF ONLY", W*0.88, H*0.62, W/2, H/2);
});
signPanel(w, h, tex, x, y, z, ry, lit);   // a plane with polygonOffset; lit = unlit basic material
```

`fitText` (Oswald) and `fitSerif` (Georgia) shrink the font until the words fit, and clamp every request so lettering never comes within 8% of the board's edge and is never taller than 62% of it. Sign faces use `polygonOffset` (factor −2, units −4) so they win the depth test against the board behind them at any distance. Lit faces are unlit basic materials; the neon panels registered in `NEON` run from 0.40 by day to 1.0 at night and stop there, because with tone mapping off anything above that clips and the lettering goes with it.

IMAGE SLOTS

Printed things such as brochures, postcards, framed prints, TV screens and vending fronts are *slots* with a fixed aspect ratio (brochure 4:9, postcard 2:3, wide print 2:1, TV 4:3, vending front 1:2). Each one draws canvas art as a fallback, and if a URL is set against its key in `IMAGES` the hosted picture replaces it at run time, provided the host allows cross-origin reads. GIFs are decoded by a small LZW decoder in `01-helpers.js`, because browsers do not animate a GIF that is used as a WebGL texture. Procedural TV static runs underneath, so a set is never a frozen frame.

MATERIALS

Almost everything is `MeshStandardMaterial` with metalness near zero. Roughness carries the material.

FAMILY	ROUGHNESS	EXAMPLES IN THE GAME
Matte: plaster, fabric, sand, carpet	`0.85 – 1.0`	`rug` 0.97 · `bedding` 0.98 · `rock` 0.97 (flat-shaded) · sun-killed paint 0.95
Satin: wood, painted block, vinyl	`0.60 – 0.85`	`block` 0.62 · `plank` 0.72 · `atomic` wallpaper 0.85
Gloss: glaze, ice, lacquer	`0.40 – 0.55`	`glazed` brick 0.42 · `ice` 0.45 · `dinertile` 0.52
Mirror-like: glass, water, chrome	`0.06 – 0.10`	`glass` 0.07 (opacity 0.34, double-sided) · `puddle` 0.06 · `mirror` 0.08
Metals	`0.30 – 0.60`	metalness 0.35–0.75 only for chrome, brass, stainless quilting and car paint

RULE · NOTHING MAY CLIP TO WHITE

Painted hexes above about `#c0` under full sun leave the shader past 1.0. The shoulder catches most of it, but keep sunlit paint at or below roughly `#d8`, give glowing materials a dark diffuse, and keep bright things such as ice, snow, porcelain and white paint a step below white. A bright surface keeps its edges only if it has room above it.

BUCKETS: HOW A WORLD BECOMES A FEW HUNDRED DRAW CALLS

Static geometry is never added to the scene directly. It is pushed into a named *bucket*; at the end of the build, `bakeBuckets()` merges each bucket into one `BufferGeometry` (positions, normals, UVs and vertex colours) with one material. The last smoke test counted 564 meshes for the whole world, including doors, cars and sprites.

```
// a bucket is a name and a material factory, declared once
bucketOf("plank", ()⇒new T.MeshStandardMaterial({color:0xffffff, map:TEX.plank, roughness:0.72}));

// then anything, anywhere, pushes into it
push("plank", planeGeo(8, 8.4, 0.5), x, y, z, ry, "#9a6a44", -Math.PI/2, 0);
bx("block", w, h, d, x, y, z, 0.833, 0, "#d8d5ca");     // a box: centre x,y,z, then size
cyl("metal", rTop, rBot, h, seg, x, y, z, "#4a4e50");

// ...and at the very end, 15-bake.js:
const BAKED = bakeBuckets(scene);             // one mesh per bucket, plus a no-cast mesh for flats
buildColGrid();                                          // everything solid is in place by now
```

`push(name, geo, x, y, z, ry, color, rx, rz)` composes the matrix with Euler order **YXZ**, so a local tilt (`rx`, `rz`) is applied before the turn about the vertical (`ry`). `y` is always the *centre* of the piece. Things that move or change (door leaves, cars, the pool water, sprites, sign faces) are ordinary meshes outside the buckets.

THE LENS

three.js fixes the *vertical* field of view and lets the horizontal follow the window's shape. At the original 72° vertical a widescreen window was 104° across, and a flat projection stretches anything off-centre by $1/\cos^2$ of its angle, so a car visibly changed shape as you turned your head. The engine now aims for 88° across and clamps the vertical between 50° and 66°. The mouse wheel only offsets that (24° tighter, 6° wider).

```
function baseFov(aspect){
  const h = 88 * Math.PI/180;
  const v = 2*Math.atan(Math.tan(h/2) / Math.max(aspect, 0.2)) * 180/Math.PI;
  return Math.max(50, Math.min(66, v));
}
```

MODELLING AT THIS FIDELITY

Objects are built from boxes, cylinders, spheres, tori and a few extrusions. At this level of detail the silhouette is everything.

01 **Model the thing that has a silhouette.** Four correctly proportioned club chairs read as "a pile of shapes". A wing chair, whose back is twice the height of its arms with wings coming forward, read at once. When a model will not read, choose a different object rather than refining the same one.

02 **Upholstery is not made of boxes.** The parts you sit on are squashed spheres, arms are capsules, and only the hidden frame is boxes. An arm 25 cm above the cushion is an arm; 7 cm is a rail. The seat tucks under the arms rather than butting them.

03 **Chain anything made of tube.** Ladders, rails and handrails are built so each piece starts where the last ended, with a quarter-torus at every bend. Built that way they cannot come apart.

04 **Know each primitive's axis.** A cylinder is born along +y and a torus along +z, so the same rotation points them different ways. Work out the rotation for every piece; a back roll given `rx` instead of `rz` became a 74 cm log through a chair.

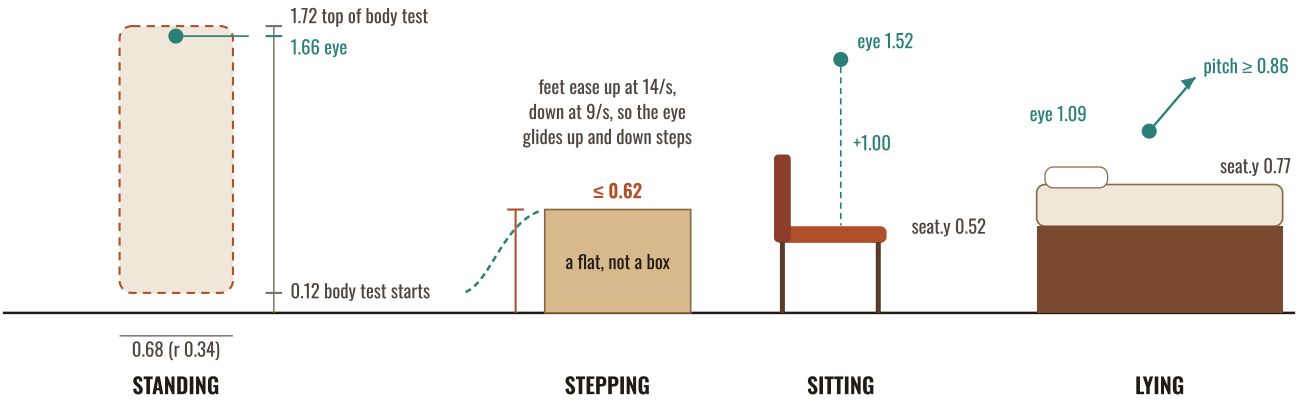
05 **A thing seen from a mile away gets one layer.** Detail you cannot resolve averages into noise. A fire road seen from the valley is one strip, one colour, one width.

06 **Render it lit, close and from four sides before believing it.** A walkthrough screenshot shows one angle, and one angle cannot tell a wrong shape from a wrong transform.

CHAPTER THREE

3 WALKING, RUNNING, SPRINTING, SITTING

The player is not a physics body. It is a point with a radius, a pair of feet that follow the floor, and an eye 1.66 m above them. That simplicity is deliberate: it makes movement predictable, makes stairs and kerbs free, and means the player can never fall through the world or get stuck in it.



Heights in metres above the floor. The dashed box is the body that collision tests: radius 0.34, from 12 cm above the feet to 1.72 m. Anything lower than 12 cm, such as a kerb or a threshold, never blocks you.

THREE GAITS

GAIT	KEY	SPEED	NOTES
Walk	W A S D or arrows	`3.05 m/s`	A brisk walk; across the motel lot in about 30 s
Run	hold Shift	`5.6 m/s`	Read by physical key code, never by `getModifierState` (on some layouts "/" needs Shift and became a run key)
Sprint	hold Q	`10.4 m/s`	For crossing the desert; nothing else reads `KeyQ`, so it cannot clash with a modifier
Wading	(automatic)	`× 0.45`	Feet below the waterline *and* inside the pool's rectangle; a blue overlay shows when the eye is under

THE MOVEMENT STEP

Each fixed step turns the keys into a direction relative to the view, moves on each axis separately so you slide along walls instead of sticking to them, and then lets the feet chase the floor.

```
const f = (keys.KeyW||keys.ArrowUp ?1:0) - (keys.KeyS||keys.ArrowDown ?1:0);
const r = (keys.KeyD||keys.ArrowRight?1:0) - (keys.KeyA||keys.ArrowLeft?1:0);
let sp = (keys.KeyQ ? SPRINT : (keys.ShiftLeft||keys.ShiftRight) ? RUN : WALK) * (underwater ? 0.45 : 1);
if(f || r){
  const len = Math.hypot(f, r), s = Math.sin(player.yaw), c = Math.cos(player.yaw);
  const dx = (-s*f + c*r)/len*sp*dt, dz = (-c*f - s*r)/len*sp*dt;   // forward is -z at yaw 0
  if(!blocked(player.pos.x+dx, player.pos.z, player.feetY)) player.pos.x += dx;   // slide on x…
  if(!blocked(player.pos.x, player.pos.z+dz, player.feetY)) player.pos.z += dz;   // …then on z
  player.step += sp*dt;
  player.bob = Math.sin(player.step*3.1) * 0.025;                    // ±2.5 cm, tied to distance
} else player.bob += (0 - player.bob)*Math.min(1, dt*6);
const target = surfaceY(player.pos.x, player.pos.z, player.feetY);
const rate = target > player.feetY ? 14 : 9;                         // step up snappily, drop softly
player.feetY += (target - player.feetY)*Math.min(1, dt*rate);
```

EYE LEVEL

The camera is placed once per frame from the player's state. The eye never has its own height; it is always a fixed distance above whatever the feet are on, which is why walking up a stair, onto a balcony or down a mine shaft needs no special code.

```
const eyeY = player.feetY + EYE + player.bob - player.sit*sitDrop;   // EYE = 1.66
camera.position.set(player.pos.x, eyeY, player.pos.z);
const cp = Math.cos(player.pitch);
camera.lookAt(player.pos.x - Math.sin(player.yaw)*cp,
              eyeY + Math.sin(player.pitch),
              player.pos.z - Math.cos(player.yaw)*cp);
```

Looking. Pointer lock on click, with drag-to-look as the fallback and touch drag on phones. Yaw changes by 0.0026 rad per pixel and pitch by 0.0024, with pitch clamped to ±1.15 rad (±66°). Keys are tracked by `e.code` in a `keys` map and all released on `blur`, `contextmenu` and when the tab hides, because the key-up for a key held while alt-tabbing never arrives.

WHAT THE FEET STAND ON: `SURFACEY(X, Z, CURRENTY)`

1. **Start from the ground.** If the point is inside a *void* (a tunnel or room dug into a hill) at the current height, the ground is that void's floor. Otherwise it is `groundY`: the paved pad at 0, the pool basin, or `Terrain.groundAt(x,z)`.
2. **Take the highest walkable surface you can reach.** Walk the list of flats and ramps; any whose height is above the current best and no more than `STEP_UP` (0.62 m) above where you already are wins.

Because the test is against where you *already* are, standing on a hill above a tunnel gets you the hill, and walking into the tunnel's mouth gets you the tunnel. Bounds tests are strict, so two flats that only meet leave a line belonging to neither; **grow every flat 10 cm into its neighbours**, and make neighbouring voids overlap by 40 cm.

```
addFlat(x0, x1, z0, z1, y);                        // a level floor, deck, stair tread or threshold
addRamp(x0, x1, z0, z1, "z", a0, a1, y0, y1);      // y runs y0→y1 as z runs a0→a1
addVoid(x0, x1, z0, z1, y0, y1, floorY);           // in this box the ground is floorY, not terrain
```

RULE · YOU STAND ON THE TRIANGLES THAT ARE DRAWN

Terrain is an analytic function drawn by sampling it on a grid and joining the samples with flat triangles. On a steep flank a 6 m cell can sit metres away from the function, so a player walking at the function's height sinks into the hill they can see. `Terrain.groundAt` finds the finest ring that actually draws a triangle at that point and interpolates within it. Use it for the player, the cars and anything placed on the ground.

COLLISION THAT NEVER TRAPS YOU

Colliders are axis-aligned boxes with a vertical span, `{x0,x1, z0,z1, y0,y1}`, so the same footprint can block the ground floor and leave the balcony above it open. `blocked(x, z, feetY)` grows each box by the player's radius and tests the body column from `feetY + 0.12` to `feetY + 1.72`.

- **Static colliders** go into a uniform 12 m grid once the world is built, padded by 1.4 m so a query near a cell edge still sees a box just over the line. Anything spanning more than 64 cells goes in a short list that is always checked.
- **Moving colliders** (parked cars) live in `COL_DYN`, outside the grid. Highway traffic has its own list that only cars test, because a pedestrian held still inside a passing truck reads as being stuck, not hit.
- **Never stuck.** If the player's own position is ever blocked (a door swung onto you, you stood up into a table), a ring search finds the nearest free spot within 6 m at a height you could have reached, and puts you there. It runs only while you are actually stuck.

```
function nearestFree(x, z, base, maxR, tol){         // rings 0.45 m out, 0.35 m apart
  for(let r = 0.45; r <= (maxR||4.2); r += 0.35){
    const n = Math.max(8, Math.round(r*14));
    for(let k = 0; k < n; k++){
      const a = (k/n)*Math.PI*2 + r;                   // each ring rotated a little
      const nx = x + Math.cos(a)*r, nz = z + Math.sin(a)*r;
      const fy = surfaceY(nx, nz, base + 0.45);
      if(Math.abs(fy - base) < (tol||1.3) && !blocked(nx, nz, fy)) return [nx, nz, fy];
    }
  }
  return null;
}
```

SITTING AND LYING DOWN

A seat is a registration, not an object: a point on the cushion, a height, the direction the sitter faces, and a name for the prompt. Any piece of furniture becomes sittable with one line.

```
addSeat(x, z, y, yaw, label, lie);
// x, z   the middle of the cushion          y     the cushion's height (a chair ~0.50, a bed ~0.77)
// yaw    the CAMERA yaw when seated         label shown in the prompt: "C · SIT · THE OBSERVER'S CHAIR"
// lie    true for a bed or a couch you lie on
```



Standing in the observation room, eye at 1.66 m. The 1955 set is visible through the glass.



Seated in the observer's chair after pressing C. The view drops, the yaw snaps to the seat's, and the mouse still looks around.

HOW IT BEHAVES

01 **Finding a seat.** **C** takes the nearest seat within 2.0 m horizontally and 1.7 m vertically. Sitting has its own key because chairs and doors are often both in reach, and when they shared **E** the door won.

02 **Settling.** Your yaw snaps to the seat's, and position and feet ease onto the cushion at rate 7 per second while the `sit` factor eases to 1 at rate 5. The eye ends at `seat.y + 1.66 - sitDrop`, with `sitDrop` 0.66 for sitting and 1.34 for lying. Lying also tips the view up to at least 0.86 rad, because on your back you look at the ceiling.

03 **Looking.** While seated the mouse looks anywhere; movement keys do nothing until they have been released once. You almost always arrive at a chair still holding W, and without that guard you would stand up on the very next step.

04 **Standing up.** Any movement key (or C again) stands you up. The engine tries spots in a fan in front of the seat, at 0.95, 1.35 and 0.65 m (1.45 to 2.25 m for a bed), sweeping ±0.52 rad steps either side of the way you faced. If every one is blocked it falls back to the ring search, so you can never be left standing inside the furniture.

RULE · A SEAT LOOKS THE OTHER WAY FROM ITS CHAIR

A chair is modelled facing its own +z and placed with `ry`, so it faces world `(sin ry, cos ry)`. The camera looks down its own −z, along `(-sin yaw, -cos yaw)`. So the seat that looks the way the chair faces has **yaw = ry + π**. Aim a chair at what it faces with `ry = atan2(tx - cx, tz - cz)`, then add π for the seat. Getting this backwards shipped three chairs facing the wall.

TUNING NOTE

With `sitDrop` at 0.66 the seated eye is 1.00 m above the cushion, which is about 1.5 m above the floor on a dining chair. A real seated eye is nearer 0.75–0.80 m above the cushion. If your world is about sitting, at a bar, in a car, in a theatre, raise `sitDrop` to about 0.9 for a lower, more natural seated view, and check that tables still clear the eye.

INTERACTION AND PROMPTS

One prompt line under the crosshair lists what you can do from where you stand, and each action has its own key so prompts never compete:

KEY	ACTION	REACH
E or Space	Get in or out of a car	within 3.6 m of a car, 2.2 m vertically
E or Space	Open or close the nearest door	within 3.4 m, same floor (1.8 m vertically)
C	Sit, lie down, stand up	within 2.0 m of a seat
F	Fly the drone (a chase-camera quadcopter)	anywhere on foot
L	Flashlight	anywhere

Zones name places on the HUD. `addZone(x0,x1, z0,z1, y0,y1, name, indoor)` registers a box, and the last zone listed that contains the camera wins, so put small rooms after the big areas around them. `indoor` marks a roof overhead: blown sand and lot dust are drawn as a field around the camera and switch off indoors, or the storm comes inside with you.

CHAPTER FOUR

4 DOORS AND DOORWAYS

The world has forty-six working doors, and almost every one of them shipped blocked at least once: by a wall course nobody split, a lintel collider hanging at head height, a sign screwed to the leaf's centre line, a face left across an opened corridor, or a sofa between the door and the rest of the room. A door is two separate things: the *leaf*, which is easy, and the *hole*, which is where everything goes wrong.



Painted panel door, motel room, opening inward onto a lit room.



Steel door into the warehouse break room, with the sign on the wall above the head.



STAFF ONLY, cut through the rink's 0.5 m block wall: two jambs, a lintel, a frame.

THE LEAF

```
makeDoor(xf, lx, ly, lz, label, opensIn, ajar, style);
// xf       a local frame from XF(ox, oz, ry): the hinge is at local (lx, lz), the leaf extends +x from it
//          (ry = 0: leaf runs along world +x · ry = π/2: leaf runs along world -z)
// ly       the bottom of the leaf, 2 cm above the floor
// label    the prompt text ("E · STAFF ONLY"); a 3-digit label reads "ROOM 107"
// opensIn  which way it swings; ajar  how open it rests (0–1); style  which leaf
```

Each leaf is its own small merged mesh inside a `Group` whose origin is the hinge, so opening it is one rotation: `g.rotation.y = base + swing × open`, with `swing` = ±1.52 rad (87°). Every face detail (panels, kick plates, push bars, louvres) is built on *both* faces of the leaf, each layer at its own depth, so it never flickers.

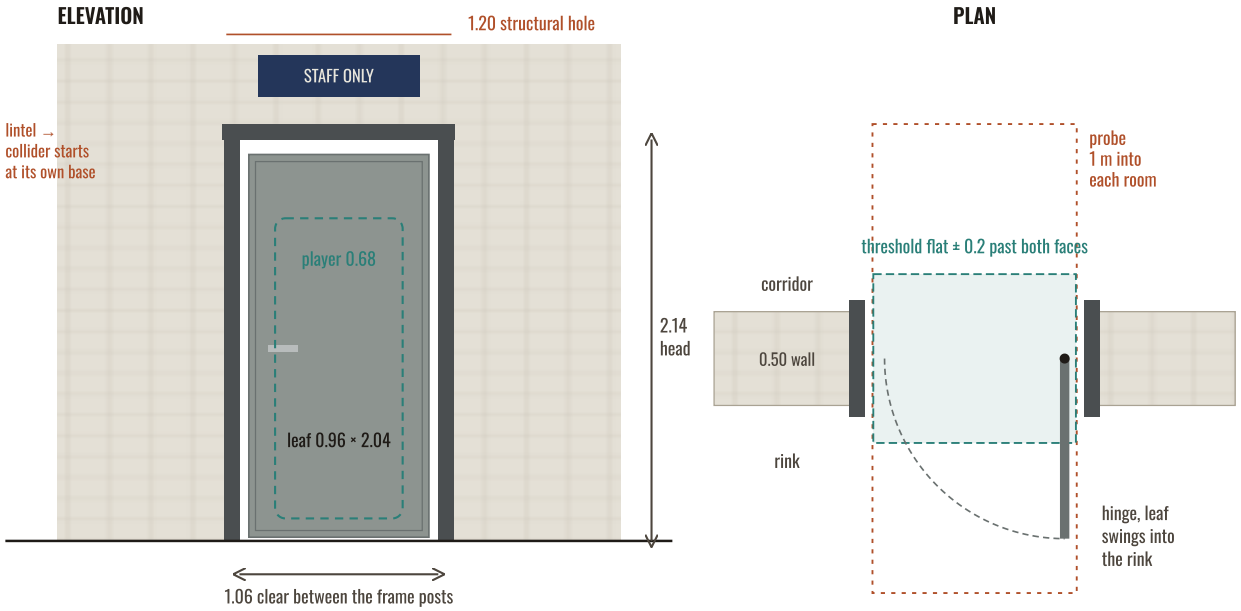
STYLE	LEAF (W × H × T)	CHARACTER
(default)	`0.92 × 2.02 × 0.055`	Motel room: cream slab, teal panels both faces, brass knob
`bar`	`0.98 × 2.02 × 0.055`	Oxblood steel, brass kick plate, push bar, porthole
`booth`	`0.98 × 2.02 × 0.055`	Olive louvred utility door with a hasp
`swing`	`0.98 × 2.02 × 0.055`	Kitchen swing door: porthole, push plate, no latch
`steel`	`0.96 × 2.04 × 0.065`	Flush grey slab, foot louvre, warning plate, lever
`deco`	`1.06 × 2.16 × 0.085`	Black lacquer, brass fluting and sunburst, speakeasy grille
`gate`	`1.42 × 1.94 × 0.045`	Galvanised frame, chain-link mesh, barbed wire, padlock

BEHAVIOUR

- **Doors open themselves.** On the same floor (feet within 1.8 m of the door's height) and within 2.5 m, a door's target is open; otherwise closed. Openness eases towards its target at rate 5.2 per second.
- **E overrides.** Pressing E on the nearest door (within 3.4 m) forces it open or shut until you walk more than 4.6 m away, after which it goes back to automatic.
- **`ajar` is a minimum.** A door with `ajar = 0.34` never closes past a third open, like the room at the far end of the walkway with the light on and nobody in it.
- **The leaf has no collider.** A door can never trap the player or hold them in a doorway, which means the opening itself must be kept clear of everything else.

THE HOLE

Every doorway in the newer parts of the world is built from the same recipe. Here it is as the STAFF ONLY door through the rink's east wall: a 0.50 m painted block wall with a 2.14 m door head.




```
const AD = ARENA_DOOR, AD0 = AD.z - 0.54, AD1 = AD.z + 0.54;         // a 1.08 m opening
wall("block", RX1, RX1+RWT, LZ1,      AD0-0.06, -0.1,      RH+0.4);   // jamb wall, one side
wall("block", RX1, RX1+RWT, AD1+0.06, RZ1+RWT,  -0.1,      RH+0.4);   // jamb wall, the other
wall("block", RX1, RX1+RWT, AD0-0.06, AD1+0.06, AD.h+0.06, RH+0.4);   // the lintel, over the head
// frame posts and head trim, proud of both faces of the wall
for(const z of [AD0-0.03, AD1+0.03])
  m.B("metal", RWT+0.12, AD.h+0.08, 0.08, RX1+RWT/2, FY+(AD.h+0.08)/2, z, 0.4, 0, "#4a4e50");
m.B("metal", RWT+0.14, 0.08, AD1-AD0+0.16, RX1+RWT/2, FY+AD.h+0.04, AD.z, 0.4, 0, "#4a4e50");
// the threshold flat, overlapping both floors, and the floor finish through the opening
m.flat(RX1-0.2, RX1+RWT+0.2, AD0, AD1, FY);
m.P("lino", planeGeo(RWT, AD1-AD0, 0.6), RX1+RWT/2, FY+0.006, AD.z, 0, RUBBER, -Math.PI/2, 0);
// the leaf, hinged 6 cm inside the far post, and the sign on the wall above, not on the leaf
makeDoor(XF(W(RX1+RWT/2), Z(AD1-0.06), Math.PI/2), 0, GY+FY+0.02, 0,
         "STAFF ONLY", false, 0, "steel");
sign(0.86, 0.22, "STAFF ONLY", "#24365a", "#e8e6dc",
     RX1-0.012, AD.h+0.34, AD.z, -Math.PI/2);
```

The `wall()` helper draws a box and registers its collider in one call. A wall that starts on the floor gets a collider sunk half a metre below it, so there is no seam to slip under. A wall that starts higher up, which is a lintel, gets a collider that starts exactly at its own base:

```
const wall = (bk, x0,x1, z0,z1, y0,y1, c, uv) ⇒ {
  m.B(bk, x1-x0, y1-y0, z1-z0, (x0+x1)/2, FY+(y0+y1)/2, (z0+z1)/2, uv||0.833, 0, c);
  m.col(x0, x1, z0, z1, FY + y0 - (y0 ⩽ 0 ? 0.5 : 0), FY + y1);     // a lintel stops where it is
};
```

THE NUMBERS THAT MAKE A DOORWAY PASSABLE

DIMENSION	VALUE	REASON
Clear width between linings	`≥ 1.00 m`	The body is 0.68 m wide; 16 cm each side lets you walk through at an angle without catching
Head height (underside of trim)	`2.14 m`	The body test stops at 1.72 m; the standard steel leaf is 2.04 m and hangs 2 cm off the floor
Lintel collider bottom	`= lintel base`	A collider sunk 0.5 m below a 2.14 m lintel becomes a bar at 1.64 m, which is head height in the doorway
Wall overlap at the jambs and head	`6 cm`	Jamb walls stop 6 cm short of the opening and the lintel spans 6 cm past it, so pieces overlap rather than meet on a plane
Frame projection	`6–7 cm each face`	Posts and trim stand proud of both wall faces, so nothing shares a plane with the wall
Threshold flat	`±0.2 m past the faces`	Flat bounds are strict; a threshold that only meets the floors leaves a line where you drop
Floor finish through the opening	`+6 mm`	Its own height, above the slab, below the rooms' finishes

RULE · CLEAR WIDTH IS THE STRUCTURAL OPENING MINUS WHAT YOU LINE IT WITH

A reveal that straddles its jamb stands into the opening on both sides. A 1.10 m structural opening with 4.5 cm of reveal each side leaves 1.01 m clear, which does not pass a 1.06 m leaf. That is a doorway blocked by arithmetic rather than by a wall.

TEN RULES, EACH PAID FOR

01 **A hole goes through every course, including the ones you cannot see.** Skirting, wainscot, dado, cap rail, plaster lining, the tile skirt outside, the concrete mass under the floor: every layer that runs along a wall with a door in it must be split around the door. The diner's front door was blocked for rounds by three decorative courses that carried no collider at all.

02 **Nothing goes between the jambs but the leaf.** Not a sign, not a hook, not a key board, not a "PRIVATE" card on the leaf's centre line. Signs go on the wall above the head, offset a centimetre along the wall's normal.

03 **A lintel stops where it is.** Only colliders that start on the floor are sunk below it.

04 **An opened end gets no face.** When a corridor that used to end in a wall is extended, the plane that closed it must go. A face has no collider behind it, so every collision probe walks straight through it while you stand looking at a grey wall.

05 **Probe a metre into each room, not just through the wall.** A plaster lining stands clear of the wall on the room side, so a test the thickness of the wall misses it.

06 **Test the drawn world, not only the colliders.** Geometry and collision disagree constantly. When a probe says a way is open and a screenshot says it is shut, the probe is testing the wrong thing.

07 **Getting to the door is part of the door.** A doorway can pass every test and still be unusable because the route to it runs through a 20 cm slot between a desk and a soda machine. Flood-fill the whole floor from the building's front door every time a door goes into a furnished room.

08 **Lay out rooms from a plan with a named lane.** Put a block of named anchors at the top of the room's code, including `LANE`, the strip from the door into the room that nothing may enter.

09 **A leaf never swings over a stairwell.** If a door opens onto a flight, hang it to open the other way. The landing starts at the threshold.

10 **Put the lights where the door leads.** A lamp's `vol` decides where it shines. A room seen through an open door is lit only if its lamps' volumes include the place you are standing.

HOW DOORS ARE CHECKED

PROBE	WHAT IT PROVES
`doors18`	Walks every doorway's lane through the colliders at ankle, waist and head height
`heads46`	Looks for any collider between ankle and head height inside each opening
`clear36` / `sight46`	Tests the *drawn* triangles of every baked bucket against a prism through the opening (a separating-axis test, so a plane crossing the hole with all its corners far away is still caught), and casts rays along both lanes naming whatever they hit by bucket and source line
`reach33`	Flood-fills a building's floor from its front door and lists the rooms you cannot get to
`walk39`	Walks a polyline the way the player does and prints surface height and zone at each leg

CHAPTER FIVE

5 THE DAY/NIGHT CYCLE

One number, the time of day, drives the sun, the moon, the sky, the stars, the fog, every lamp, every glowing surface, every lit sign, the pool, the headlights and the wildlife. Nothing else in the world knows what time it is; it only asks how dark it is.



12:30. Sun 1.12, sky fill 1.02, lamps off.



17:24. The key turns orange; lamps begin to ramp up.



18:30. Twilight: the key at its weakest (0.16), every lamp fully on.



22:00. Moonlight 0.38 from the blue key, stars wheeling, haze darker than the sky.

THE CLOCK

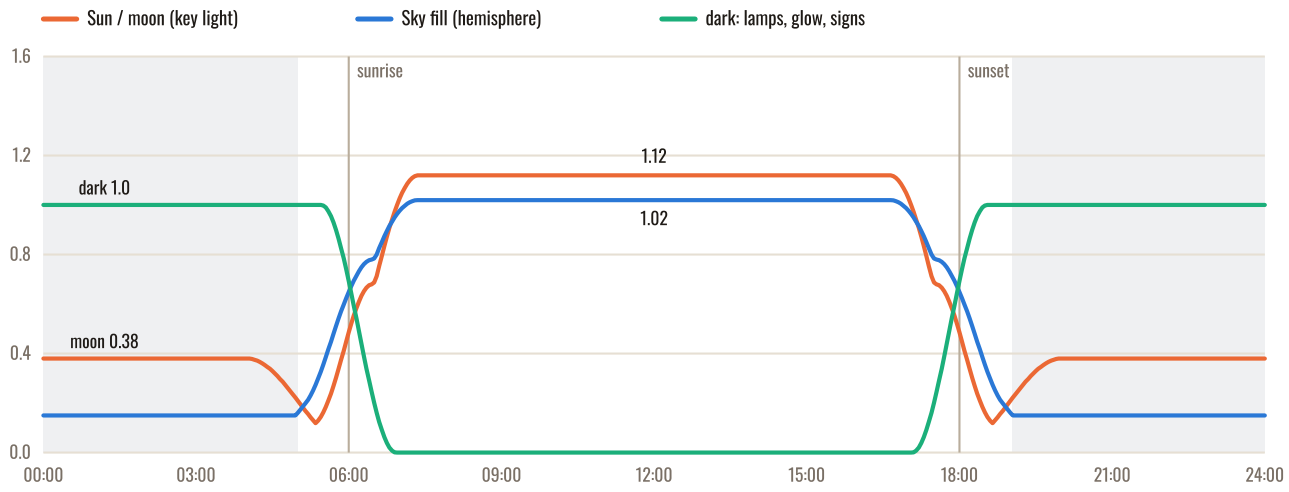
```
const DayNight = { DUR: 414,                          // real seconds for 24 game hours
  state: { t: 0.478, ... },             // t ∈ [0,1); 0 is 06:00, so the game opens near 17:28
  setHour(h){ s.t = (((h - 6)/24) % 1 + 1) % 1; },
  nudge(d){ s.t = (s.t + d + 1) % 1; },
  tick(dt){ if(!s.paused) s.t = (s.t + dt/DUR) % 1;   ...   s.hour = (s.t*24 + 6) % 24; }
};
```

The player can jump to nightfall (**N**, 21:36) or midday (**M**, 12:30) and scrub time with **[** and **]** (about 29 game minutes a press). Before the player clicks *start*, the clock runs at a quarter speed behind the title card.

FROM THE CLOCK TO THE LIGHT

The sun travels a circle: `ang = t × 2π`, and its height is `sunHeight = sin(ang)`, which is 0 at 06:00 and 18:00, 1 at noon and −1 at midnight. Every blend in the cycle is a `smoothstep` of that one value:

QUANTITY	FORMULA (SH = SUNHEIGHT)	DRIVES
`day`	`smoothstep(0.12, 0.35, sh)`	full daylight weight
`set`	`smoothstep(-0.22, 0.12, sh) × (1 - smoothstep(0.15, 0.35, sh))`	the orange band either side of the horizon
`nightAmount`	`1 - smoothstep(-0.50, -0.02, sh)`	sky, key and haze colours; stars; clouds
`sunsetAmount`	`set + 0.6 × twilight`	how orange the key light is
`dark`	`1 - smoothstep(-0.14, 0.24, sh)`	**every artificial light**: lamps, glow, signs, headlights
Key intensity	`max(0.04, 1.12·day + 0.68·set) + 0.34·night`	sun by day, moon by night
Sky fill intensity	`max(0.15, 0.70·day + 0.46·set + 0.32·(1 - night))`	the hemisphere light



Light levels over 24 game hours, computed from the formulas above. The shaded bands are full night. The key light is weakest at about 18:30, between the sun going down and the moon taking over, and that dip is what makes dusk read as dusk. Lamps (`dark`) ramp up between about 17:15 and 18:30 and fade out between 05:30 and 07:00.

WHERE THE SUN IS

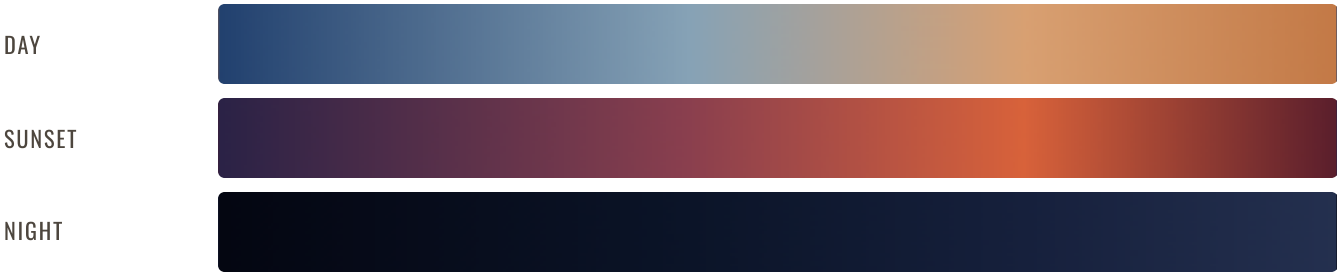
```
sunDir.set(Math.cos(ang)*190, Math.max(14, Math.sin(ang)*190 + 6) - camY*0.3, 70).normalize();
// rises in +x (east), sets in -x (west), always tilted towards +z (south) by the 70
// never lower than 14 above the horizon as a light, so shadows never stretch to infinity
placeCelestial(sunDisc,  ang,  1, 70/190);     // sprites 1,800 m out along the same direction…
placeCelestial(moonDisc, ang, -1, 70/190);     // …and the moon exactly opposite
```

The sun disc is an additive sprite that grows from 320 to 580 units and shifts from `#fff4d6` to `#ff6f28` as it nears the horizon, fading out as it drops below. The moon fades in with `nightAmount`. Stars are three `Points` layers by magnitude, with colour temperatures from blue-white to orange and a Milky Way band, hung on a tilted pivot that turns with the hour, so the sky wheels overhead.

THE PALETTE

Every colour the cycle uses lives in one object, `DayNight.P`. To move this engine to a different climate (a grey northern coast, a green valley, an alien planet) this is the first thing to repaint.

SKY GRADIENT: ZENITH → MID → LOW → HORIZON



Each frame `paint()` mixes the three sets by day, set and night weights and redraws a 16 × 256 canvas with seven stops (0, 0.19, 0.33, 0.42, 0.58, 0.72, 1.0). The canvas maps onto a 2,400 m sphere drawn inside-out around the camera, with fog and depth writing off. The horizon colour is also the fog colour and the clear colour, so the world's edge always dissolves into the sky.

LIGHTS

`key · day #ffeed0` `key · set #ff9b4c` `key · night #8ea6d4` `sky fill · day #ffe7c4`
`sky fill · night #18243a` `ground bounce · day #4a2a18` `ground bounce · night #080a12`
`night haze #0a1020`

```
key.color.copy(P.dayLight).lerp(P.setLight, Math.min(1, dn.sunsetAmount*1.2))
         .lerp(P.nightLight, dn.nightAmount);
hemi.color.copy(P.dayHemi).lerp(P.nightHemi, dn.nightAmount);
hemi.groundColor.copy(P.dayGround).lerp(P.nightGround, dn.nightAmount);
amb.intensity = 0.15 + dn.nightAmount*0.06;
scene.fog.color.copy(skyImg.hz).lerp(NIGHT_HAZE, dn.nightAmount*0.72);
scene.fog.near = 185 + dn.nightAmount*52;
scene.fog.far  = (1460 + dn.nightAmount*330) * (1 - Wind.gust*0.46);
```

WHAT `DARK` SWITCHES ON

SYSTEM	RESPONSE	NOTE
Pool point lights, outdoors	`intensity × dark`	Porch lights, lot poles, sign floods
Pool point lights, indoors	`intensity × max(dark, 0.70)`	Drapes drawn: rooms are lamplit all day
Pool point lights, underground	`intensity × 1`	No day forty feet down
Emissive surfaces (`GLOW`)	`k = 0.10 + 3.0·dark`	`"pool"` kind 0.30 + 2.0·dark; `"buried"` fixed at 2.35; all through the soft knee
Unlit glow quads	`opacity = dark (or 1 - dark, or 1)`	Flags `day` , `always` , `max` ; `blink` and `duty` make a beacon
Neon panels (`NEON`)	`0.40 + 0.60·dark`	Stops at 1.0, or the lettering clips away
Your car's headlights	`on above dark 0.12`	Ramp to full by about dark 0.45
Traffic lamps	`visible above dark 0.04`	Lens dark × 0.98, halo dark × 0.34
Pool water shader	`sun strength, glow, reflection, foam`	Less mirror and more underwater glow at night
Wildlife and weather	`by nightAmount`	Birds and the vulture leave at night; moths circle outdoor lamps after dark; lot dust stops

RULES FOR THE CYCLE

- **Ask how dark it is, never what time it is.** Everything reads `dark` or `nightAmount` . Change the curve in one place and the whole world follows.
- **Twilight lingers.** A desert dusk is a long warm band, not a switch. The `set` and twilight terms overlap the horizon on both sides so the change takes about an hour of game time, about 17 real seconds.
- **Turn off shadows you cannot see.** Below a sun height of 0.04 the key stops casting; the moon's shadows would be all acne and no shape.
- **Give the night a real moon.** A key of about 0.38 in pale blue is enough for cliffs and strata to keep reading across the valley. A token moon gives you a black screen with lit windows in it.
- **Haze is darker than the night sky.** Otherwise distant mountains wash out into the fog instead of standing against it.



The court at 13:00. Sun and sky do everything; every lamp is off.



The same court at 22:00. Lit windows are glowing surfaces, porch lights are pool lamps, and the pool glows from its own shader.

CHAPTER SIX

6 CARS

Seventeen parked cars, every one of them drivable, and sixteen vehicles on the highway that react when you hit them. The car engine splits the problem the useful way: four springs own everything vertical (bounce, pitch, roll), and a two-axle tyre model owns everything flat (grip, slide, yaw). The two never argue because the tyre model never touches height.



A parked sedan. About fifty boxes and cylinders in two merged meshes (paint and glass) plus four wheel meshes that steer, roll and follow the springs.



Driving at night. The chase camera, the tail lamps, and two converging headlight beams lighting the lot and the motel front.

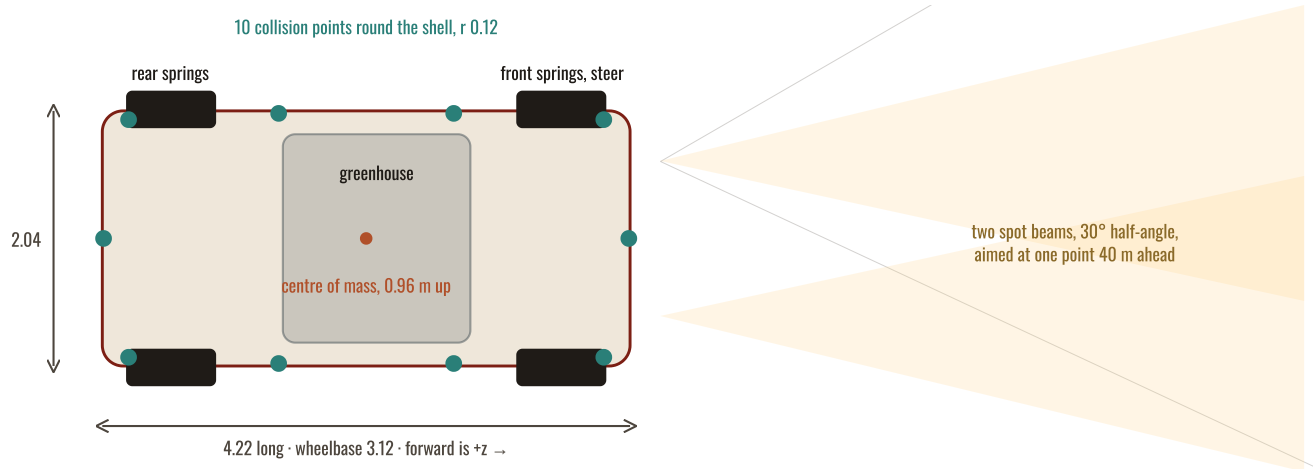
HOW THE CARS LOOK

A car is modelled in its own space (x right, y up, z forward, origin at the centre of mass) from about fifty primitives, each with a hex colour, then merged. Three body styles share one lower body and differ above the beltline.

PART	SIZE (M)	DETAIL THAT SELLS IT
Rocker and pan	`2.04 × 0.30 × 4.22`	Near-black, so the body seems to float over its own shadow
Main body	`1.98 × 0.52 × 4.10`	The paint colour; top at 0.93 m above the ground
Hood deck, trunk deck	`1.82 wide`	Separate boxes, each at its own height, so they never share a plane
Bumpers	`2.06 × 0.28 × 0.22`	Chrome `#b8b0a0`, slightly wider than the body
Grille, plate, lamps		Round headlamps (r 0.125) in chrome bezels, red tail lamps, a pale plate
Shut lines and handles	`0.02 grooves`	Dark lines where the doors are and chrome handles: what tells you where a car comes apart
Greenhouse	per style	Sedan: raked windshield and backlight (0.42 and −0.52 rad), roof cap, mirrors, antenna. Pickup: cab, bulkhead, bed with slats. Van: tall box, rear doors

Arches	`0.10 × 0.30 × 1.14`	A dark box inside each arch and a fender lip: wheel wells read as depth
Wheels	`r 0.36, 0.30 wide`	Tyre, white wall, hub, cap and two spokes, so you can see it turn

Materials. Body `roughness 0.38, metalness 0.34`; glass `opacity 0.80, roughness 0.08, metalness 0.45`; tyres carry the rubber texture at roughness 0.95. The paint palette is muted period colour: oxblood `#7c1f14`, navy `#21386e`, silver `#b0b3b8`, green `#2f5a34`, tan `#8a5a1e`, charcoal `#3a3735`, rust `#a8452b`, sage `#6d6b5e`.



THE PHYSICS

BODY AND SPRINGS

The car is one rigid body of 1,200 kg with a box inertia (2.0 × 1.0 × 4.3 m; roll stiffened × 1.4) and a quaternion orientation. Each wheel is a spring that looks straight down: it asks `surfaceY` under its mount, the same function the player walks on, so cars drive up onto kerbs and forecourts rather than through them. If the ground is within rest length plus wheel radius, the spring pushes along the car's up vector *at the wheel*. Because the force is applied at the wheel and not at the centre, braking pitches the nose down and cornering rolls the body with no extra code.

```
const disp = CARC.restLength - (len - CARC.wheelRadius);    // how far the spring is compressed
const sv = up.dot(pointVelocityAtWheel);                     // how fast it is compressing
force.copy(up).multiplyScalar(CARC.springK*disp - CARC.damping*sv);
body.addForceAtPosition(force, wheelWorldPos);               // a torque about the centre of mass
```

Ride height. At rest each spring carries a quarter of the weight, so it sags `m·g / 4k` = 8.7 cm. `CAR_RIDE = (rest - sag + radius) - mount` = 0.96 m is where the body floats, and parked cars are built at exactly that height so nothing jumps when you get in.

TYRES: THE BICYCLE MODEL

With all four wheels down, the car is treated as two axles. Each axle's sideways force comes from its slip angle, clamped at a grip limit, and the difference between the front and rear forces, acting 1.56 m either side of the centre, turns the car.

```
steerAngle = steer * (1 - Math.min(speed, 40)/40) * 0.42;   // steering washes out with speed
const rot  = Math.atan2(yawRate, |vf|), slip = Math.atan2(vl, |vf|);
const latF = clamp(-5.0*(slip + rot - steerAngle), -2.6, 2.6) * axleLoad;   // axleLoad = m·g/2
let   latR = clamp(-5.2*(slip - rot),              -2.6, 2.6) * axleLoad;
if(handbrake) latR *= 0.5;                                  // the whole of "the back steps out"
let trac = 9000*throttle;  if(braking) trac = -12000*Math.sign(vf);
const drag = -5.0*vf*Math.abs(vf), roll = -15*vf;
yawAccel = (1.56*latF - 1.56*latR) / mass;
```

CONSTANT (`CARC`)	VALUE	EFFECT
mass · gravity	`1200 · 9.81`	Sets axle load, sag and inertia
engineForce · brakeConstant	`9000 · 12000`	About 0.75 g of acceleration, a little more braking
dragConstant · rollingResist	`5.0 · 15`	Top speed where $5v^2 + 15v = 9000$: 41 m/s, about 92 mph
corneringFront · Rear	`-5.0 · -5.2`	A slightly stiffer rear: stable, mild understeer
maxGrip · maxSteer	`2.6 · 0.42 rad`	Grip limit (× axle load); full lock 24°
steerSpeedBias	`40 m/s`	Steering fades linearly to zero at this speed
restLength · wheelRadius	`0.34 · 0.36`	Suspension travel and ride height
springK · damping	`34000 · 4200`	A soft period ride that settles in one bounce

AIRBORNE, COLLISIONS AND THE GROUND

- **In the air** the tyres do nothing. Spin is split into yaw and everything else: pitch and roll bleed off fast ($\times e^{-3.6\,dt}$), yaw persists ($\times e^{-0.45\,dt}$), and a damped torque about `up × worldUp` rolls the roof back upright. W/S and A/D trim pitch and roll on the way down. Cars arc rather than tumble, and land driving.
- **Walls.** Ten points round the shell (r 0.12) are tested against the static grid, the moving colliders and the traffic. Movement is reverted per axis, x then z, with a −0.18 bounce, so the car slides along a wall the way the player does.
- **Fast slopes.** At 38 m/s a car covers two thirds of a metre a step, so a hillside can be above a wheel before its spring reacts. After every step the body is lifted out of the deepest wheel that has gone under, at up to 4 m/s.
- **No creep.** With no throttle and under 0.5 m/s, planar velocity and yaw are zeroed, so a parked car never drifts.

DRIVING IT

KEY	ACTION
E (or Space on foot)	Get in, within 3.6 m of a car. **E** again gets out on the driver's side
W	Throttle
S	Brakes while moving forward faster than 0.8 m/s, then reverses at 55% throttle
A / D	Steer, smoothed 18% of the way to the target each step, so the wheel turns rather than snaps
Space	Handbrake: halves rear grip. Space no longer means "use" once you are in a car
Mouse	Orbits the chase camera (yaw free, pitch −0.18 to 1.05 rad)

THE CHASE CAMERA

```
const sp  = Math.min(1, Math.abs(speed)/28);                  // pulls back as you speed up
const horiz = (7.6 + sp*3.0) * Math.cos(camPitch);
target.set(car.x + Math.sin(a)*horiz,
           car.y + 2.20 + (3.4 + sp*1.3)*Math.sin(camPitch),
           car.z + Math.cos(a)*horiz);
target.y = Math.max(target.y, surfaceY(target.x, target.z, target.y) + 0.9);   // never under the ground
camera.position.lerp(target, 1 - Math.exp(-10*delta));       // exponential: same lag at any frame rate
look.lerp(car.position, 1 - Math.exp(-15*delta));
camera.lookAt(look.x, look.y + 0.85, look.z);
```

RULE · DRAW THE CAR BETWEEN STEPS, AND AIM THE CAMERA AT WHAT IS DRAWN

Physics runs at 60 Hz whatever the display does. `savePrev()` keeps exactly one step of history, and `showCar()` places the shell at `lerp(prevPos, pos, stepAlpha)` and `slerp(prevQuat, quat, stepAlpha)`, with suspension travel and steer angle interpolated the same way. The camera then follows the *shell*, not the physics body; aimed at the body, the two were a fraction of a step apart every frame and the car shivered. Use $1 - e^{-k\Delta}$ rather than $k \cdot \Delta$ for every follow, or the lag changes with the frame time.

HEADLIGHTS



From above. Two spotlights from the lamp positions, converging on one point 40 m ahead, light the asphalt in front of the car.



Head-on. The lenses are small amber planes, never white: the size of the light is in the beam, not the lamp.

Real lights are expensive, so there is only one pair, made once and moved to whichever car you are driving. They switch on above `dark = 0.12` and reach full strength by about 0.45.

PIECE	SETTING	WHY
Beams	`SpotLight(0xffd68c, 1.65, 90, π/6, 0.55, 1.5)` × 2	Tungsten colour, about 2,800 K. At 2.2 the road blew out to white
Beam origin and aim	2.35 m forward, ±0.62 m across; one target 40 m ahead and 0.4 m down	The cones converge the way a real car's do
Lenses	0.30 × 0.20 planes, `#ffcf82`, opacity 0.80, unlit, tone mapping off	An unlit basic material cannot go above white, so the only way to keep a lamp warm is not to ask it to be brighter than white
Tail lamps	0.34 × 0.15 planes, `#ff2a12`, opacity 0.78	Face backwards (turned π), on whenever the headlights are

> **DESIGN NOTE**
> A headlamp is a small bright thing that throws a large dim thing. The first version was one beam plus a pair of 3 m glow sprites, and it read as headlight-shaped clouds. Keep the lens the size of a lamp and put all of the size into the light it throws.

PARKED CARS AND GETTING OUT

- Parked cars are separate groups (not baked) with an axis-aligned collider in `COL_DYN`, sized from the rotated footprint: `hx = |cos yaw|·1.03 + |sin yaw|·2.28`.
- Getting in copies the shell's transform into a fresh physics body and disables the car's own collider so it cannot block itself.
- Getting out tries the driver's side, 1.9 m left of centre, with a tight height tolerance (0.45 m) before a loose one, so you never step out onto the shoulder a metre below where you got in. Then the collider is rebuilt round wherever the car stopped.

HIGHWAY TRAFFIC

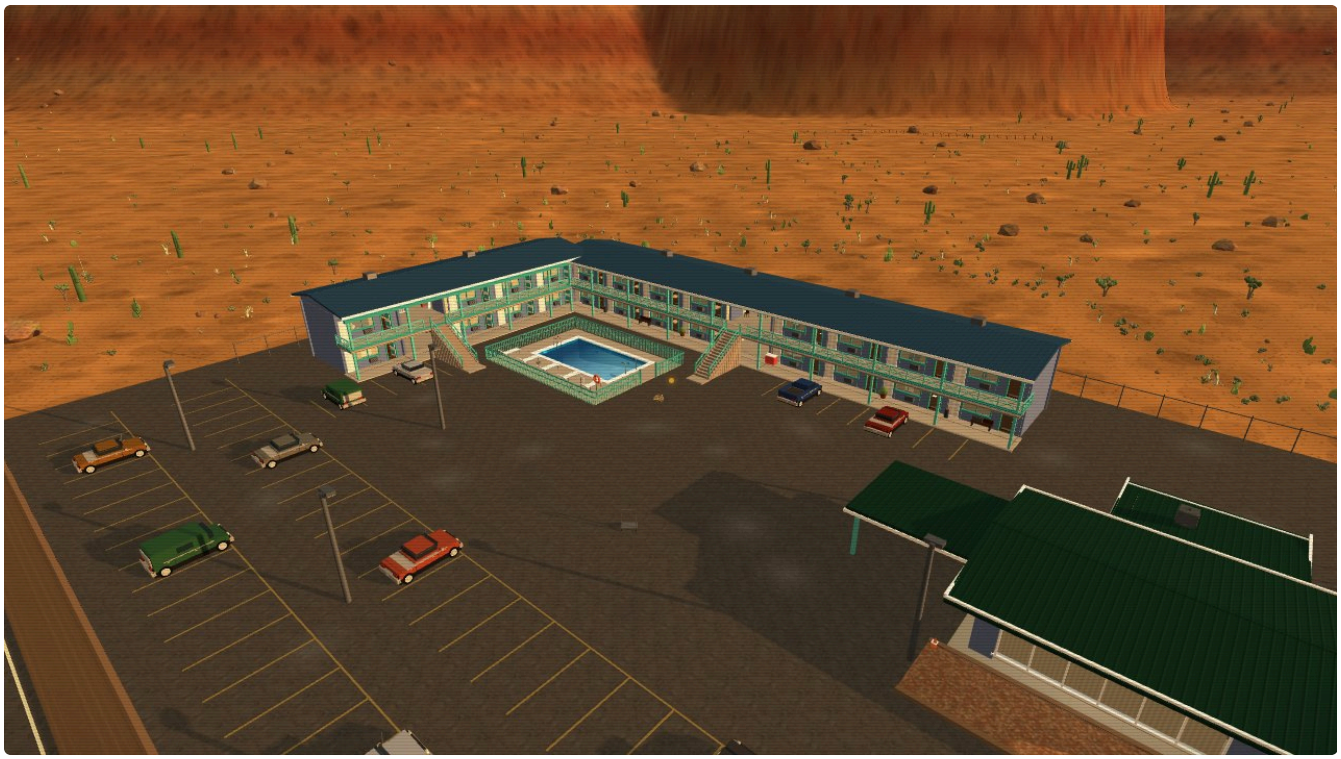
Sixteen vehicles (sedans, pickups, and tractor-trailers) run on rails in two lanes 2.2 m either side of the centre line, at 26–32 m/s for cars and 19–25 m/s for trucks, and wrap at ±1,500 m, well past the fog, so nothing is ever seen appearing. Each keeps a moving collider that only cars test. At night their lamps are small bright lens planes with a faint additive halo behind.

Being hit moves the other car too. A struck vehicle gets a sideways velocity and a speed loss scaled by how hard you hit it, divided by its weight (trucks × 1.9). A damped spring (stiffness 6.4, damping 2.3) walks it back into its lane, overshooting once, and its heading is never animated; it is simply pointed along its own velocity with `atan2(dir·speed, swayV)`. The slew, the correction and the fishtail all come from one number, so they cannot disagree with the path. Your car gets one impulse per collision, along the truck's direction and off the centre line, and is moved clear of the box if the space is free.

CHAPTER SEVEN

7 BUILDING A DIFFERENT WORLD

Everything in chapters 2 to 6 is reusable. This chapter is about the part you replace, the content, and about the conventions and habits that kept a world of this size from falling apart as it grew from a motel to a desert with a dozen buildings and four levels underground.



The motel site from the drone (F). Two wings in an L round the pool, the office, the lot and the highway. Each building is authored in its own local frame and placed by a transform.

COORDINATES AND FRAMES

- **World:** metres; `+y` up; `+x` east; `+z` south (yaw 0 looks down −z, which the compass calls north). The highway runs along x at `z = -48.5`.
- **Rotations** use Euler order `YXZ`: tilt first, then turn about the vertical. A plane's front faces +z, so a wall lining with `ry` faces `(sin ry, 0, cos ry)`; point it *into* the room.
- **Colliders are axis-aligned**, so buildings turn only in quarter turns. Anything at another angle is scenery or gets a stepped set of boxes.

THREE LOCAL FRAMES

```
const m = mk(ox, oz);              // origin at (ox, oz), heights relative to the ground there (m.y)
m.B(bucket, w,h,d, x,y,z, uv, ry, color);    m.C(...);   m.P(bucket, geo, x,y,z, ry, color, rx, rz);
m.col(x0,x1, z0,z1, y0,y1);   m.flat(x0,x1, z0,z1, y);   m.voidAt(x0,x1, z0,z1, y0,y1, floor);
m.rect(x0,x1, z0,z1, name);   m.lamp(x,y,z, {color, intensity, dist, ...});

const m2 = mkPi(ox, oz, gy);       // the same, turned 180° (bounds negate and swap), ground height given

const xf = XF(ox, oz, ry);         // any quarter turn: xf.x(lx,lz), xf.z(lx,lz)
                                   // used by lbx, lwall, lflat and makeDoor
//   x = ox + lx·cos ry + lz·sin ry      z = oz - lx·sin ry + lz·cos ry      (three.js's own handedness)
```

RULE · WRITE THE OFFSET HELPER ONCE, THE RIGHT WAY ROUND

three.js's `rotation.y = t` sends a local offset `(dx, dz)` to `(dx·cos t + dz·sin t, -dx·sin t + dz·cos t)`. Writing the 2-D rotation out of habit as `(dx·cos - dz·sin, dx·sin + dz·cos)` orients every piece by +t and positions it by −t. At t = 0 the two agree and every test passes; at any other angle the object comes apart into correctly shaped blocks in the wrong places, which looks exactly like bad modelling. It cost five rebuilds of one armchair.

AUTHORING A ROOM

01 **One page of numbers.** Open the block with every dimension named: the room's extents, wall thickness, ceiling height, each doorway's centre and half-width, and the `LANE` from the door into the room. Everything after it is written in those names, so a door can move without breaking a wall.

02 **A wall you can stand inside is a ring, not a slab.** Four thin runs, each with its own collider. A doorway is two pieces plus a head. Then name every wall out loud, in order, and check the list against the code; one corridor shipped missing nine metres of its north side.

03 **Floors: the slab's top is the flat's height.** `B()` takes a box's centre, so a 0.26 m slab under a flat at `F` is centred at `F - 0.13`. Lay the finish (lino, planks, carpet) a few millimetres above at its own height, and register one `addFlat` for the floor, grown 10 cm into every neighbour.

04 **Ceiling, then volumes.** A zone with the room's name and `indoor:true`; underground, a void for the walkable space and a buried volume with the room's ambient colour.

05 **Lamps, then glow.** Register the lamps that must light the room, each with a `vol` of the room. Then the fittings that glow: shades as open tubes, ceiling fittings as domes.

06 **Furniture from anchors, outside the lane.** Seats get `addSeat` with yaw = chair ry + π. Anything you can walk into gets a collider; nothing goes in the lane.

07 **Run the checks** (below) before calling it finished.

TERRAIN

The ground is a function, `Terrain.heightAt(x, z)`: fractal value noise for the desert floor, a ring of mesas built from radial rims, and graded *pads* blended in where something is built (the motel pad and highway corridor at −0.25, the diner, the filling station, a dish, a trail cut into a butte). It is drawn as nested level-of-detail rings: 0.5, 2, 3, 6 and 12 m cells, each ring with holes where a finer ring sits.

- **Ring cell sizes must be exact multiples** of one another, and each inner ring's box must land on the outer ring's lattice, or the rings only touch every few cells and the seams open onto sky.
- **Every ring hangs a skirt** off its edges to hide cracks between levels of detail, single-wound on a double-sided material (a skirt wound both ways sums its normals to zero and shades black).
- **Anything dug under the terrain goes in** `SKIRT_GAPS`, which shortens the skirts over that box to 1.2 m. A 50 m skirt once hung straight down through the middle of the speakeasy as a pale wall that no file drew.
- **Build on a pad, not on a thicker slab.** A site on sloping ground gets a graded pad in the height function, blended over a few metres, so the floor, the apron and the ground agree.
- **A path up a slope is cut into the ground** (a bench forced into `heightAt` along a polyline), and the ground there needs cells fine enough to show it.

GOING UNDERGROUND

Underground rooms are the engine's most fragile feature and its most striking one. The order of work that has held up:

1. **Survey first.** Print the ground profile over the site and list every existing void and buried volume. Keep two metres of rock and the slab between any ceiling and the surface.
2. **Voids end to end, overlapping by 40 cm.** A sample landing exactly on a seam between two voids belongs to neither, hands you the terrain four metres overhead, and you are stood on the roof for good, because `surfaceY` asks about the height you are already at.
3. **Buried volumes** for every room, with its own ambient colour, so the sky fades as you descend.
4. **A shaft that breaks the surface is cut out of the surface:** a hole in a fine terrain patch, plus `SKIRT_GAPS` entries for the patch's rim and the bore.
5. **Colliders that you later dig under must be asked how deep they go.** A fence registered from `y = -2` stood across a tunnel four metres down.

TWO FACES ON ONE PLANE FIGHT

The depth buffer cannot separate two surfaces at the same depth that face the same way, so they speckle as you move. Faces that merely butt edge to edge point opposite ways and one is culled, so butting is free and overlapping coplanar faces is the thing to avoid.

- A slab and its finish are never at the same height. A wall's foot sinks into the slab rather than sitting on its top. A riser's ends go *inside* the treads.

- Anything applied to a surface (a plate on a door, a stain on a wall, a worn board) sits at its own depth. Twenty-six of them are twenty-six depths, at least 0.2 mm apart up close.
- Floor decals stand on a ladder at least 0.6 mm apart, each taking the rung above the highest one already under it. The decal you want read goes last. A count and a step are one decision: 500 decals at 0.4 mm put the top one 20 cm in the air.
- Sign faces use `polygonOffset` rather than distance, because depth precision falls with the square of range.
- Make a jamb 0.11 thick and let it overlap; you never see the inside of a solid.

PERFORMANCE

TECHNIQUE	WHAT IT BUYS
Merge buckets	Tens of thousands of primitives in a few hundred draw calls. Frustum culling does the rest
Fixed light count	One shadowed key and twelve unshadowed point lights: one short shader loop that never recompiles
Flats and transparents do not cast	A smaller shadow pass and tighter bias
Shadow box follows the player	3 cm texels over 92 m instead of blurry shadows over a kilometre
Collider grid	Thousands of rocks, posts and walls; each query looks at one 12 m cell
Allocation-free hot paths	The car's `up()`, `forward()` and `right()` each own one vector and recompute it in place; about 1,400 fewer throwaway objects a second while driving
Fields around the camera	Sand, dust, clouds and birds are drawn relative to the camera and wrap, so the world feels endless at constant cost
Remembered answers	Whether a lamp is underground is asked once and cached, since volumes never move

ATMOSPHERE: WHAT MAKES IT FEEL INHABITED, AND EMPTY

Liminal spaces are mostly made of small movements in places where nobody is. These are all cheap and they do more for immersion than any amount of geometry:

Wind. One wind vector with gusts; sand particles stream along it, the fog tints with dust, and lot dust lifts by day. All of it switches off indoors.

Tumbleweeds. Seven, each grown as a tangle of forked twigs rather than a ball of spokes, rolling on a lumpy sphere so they hop.

Litter and geckos. Paper that blows across the lot now and then; geckos that freeze and dart on warm rocks.

Sky life. Clouds tinted by the sky, distant hot-air balloons that sit in the same haze as the mountains, birds by day, and one vulture that circles a thermal for a couple of minutes and then is gone.

Moths. They orbit whichever outdoor lamps (`mothy`) the light pool is currently driving, and only after dark.

Bad fixtures. A shared flicker (a product of three sines) for one lamp in each alcove; the VACANCY sign's bad tube; a TV left on behind one curtain, pulsing blue.

Screens. TVs play real GIFs when a host allows it and procedural static when not, with a scan bar rolling through.

Names. Every place names itself on the HUD when you arrive, so finding something registers as having found something.

THE HUD

Plain HTML over the canvas: a compass rose (SVG) and heading, the clock, the current zone's name, a vacancy bar, a painted site plan, a one-line prompt and short toasts. **H** hides it, **O** switches to an orthographic plan view from 210 m up. Keep it small and quiet; the world is the interface.

THE CHECKS THAT ARE NOT OPTIONAL

After four rounds of the same three faults being found in screenshots instead of by the tools, these became mandatory for anything added to the world, including "only a corridor":

01 **Coplanar scan** over the new region's box: every pair of overlapping faces on one plane, at any orientation, with the invisible ones filtered out.

02 **Door probes and a walk** through the new space and back out: colliders at ankle, waist and head height, drawn geometry along each lane, and a polyline walked the way the player walks.

03 **Name every wall** of every room out loud and check the list against the code.

04 **A free-camera shot from inside**, pitched, and for anything object-sized a close lit shot from four sides. The ray probes cannot see a wrong colour, a stretched map or a cylinder pointing the wrong way, and all three have shipped.

The probes run headless: Chromium with a software renderer loads the page, and the game exposes a `window.MOTEL` handle (move the player, step the simulation, set the hour, open doors, enter cars, read colliders and zones) for scripts to drive. A second copy of the page records every primitive's world box and source line, so a probe can say *which line* drew the thing in the doorway. Build that handle into a new world on day one.

> **RULE · WHEN A PROBE AND A PICTURE DISAGREE, THE PROBE IS TESTING THE WRONG THING**
> Geometry and collision are authored separately and drift apart. A probe walking cleanly through a wall the screenshot shows means the wall has no collider, or the probe reads colliders and the blocker has none. Find which, rather than trusting either.

APPENDIX

A REFERENCE CARD

REGISTRATION FUNCTIONS

CALL	REGISTERS
`push(bucket, geo, x,y,z, ry, color, rx, rz)`	A static piece; `y` is its centre; Euler YXZ
`bx(bucket, w,h,d, x,y,z, uv, ry, color)`	A box with world-scaled UVs (`uv` = repeats per metre)
`cyl(bucket, rt,rb,h,seg, x,y,z, color, rx,ry,rz)`	A cylinder (axis +y before rotation)
`bucketOf(name, ()⇒material)`	A bucket's material, made at bake time
`addCol(x0,x1, z0,z1, y0,y1)`	A solid box (defaults y −2 to 60)
`addFlat(x0,x1, z0,z1, y)`	A walkable level surface
`addRamp(x0,x1, z0,z1, axis, a0,a1, y0,y1)`	A walkable slope along `"x"` or `"z"`
`addVoid(x0,x1, z0,z1, y0,y1, floor)`	Inside this box the ground is `floor`, not the terrain
`addZone(x0,x1, z0,z1, y0,y1, name, indoor)`	A named place; `indoor` stops weather
`addBuried(x0,x1, z0,z1, y0,y1, amb)`	Underground: no sun, low sky, ambient `amb`, lamps off the clock
`addSeat(x, z, y, yaw, label, lie)`	Somewhere to sit or lie (yaw = chair ry + π)
`LAMPS.push({x,y,z, color, intensity, dist, decay, indoor, vol, flicker, mothy})`	A fixture the light pool may drive
`makeDoor(XF(ox,oz,ry), lx,ly,lz, label, opensIn, ajar, style)`	A swinging leaf; hinge at local (lx,lz), leaf along +x
`glowMat(opts, kind)` · `emitMat(color, k, kind)`	An emissive that ramps with `dark`; kinds `warm`, `cool`, `pool`, `buried`
`basicGlowMat(color, kind, blending)`	An unlit quad whose opacity follows `dark`; set `always`, `day`, `max`, `blink`, `duty` on its `GLOW` entry
`signTex(w,h, draw)` · `signPanel(w,h, tex, x,y,z, ry, lit)`	A canvas face and the plane that shows it
`fitText(ctx, str, maxW, size, cx, cy)` · `fitSerif(...)`	Lettering that shrinks to fit and keeps its padding
`Terrain.heightAt(x,z)` · `Terrain.groundAt(x,z)`	The field · the drawn surface (use this one)
`surfaceY(x,z,curY)` · `blocked(x,z,feetY)`	What you would stand on · whether the body fits
`nearestFree(x,z, base, maxR, tol)`	The nearest spot the player could stand

CONTROLS

KEY	ON FOOT
W A S D / arrows	Walk
Shift · Q	Run · sprint
Mouse	Look (click for pointer lock)
Wheel	Zoom the lens
E · Space	Door, car
C	Sit, lie, stand
L	Flashlight
F	Drone

KEY	WORLD
N · M	Nightfall · midday
[·]	Scrub time ±29 min
O	Plan view
H	Hide the HUD
KEY	**IN A CAR / DRONE**
W · S	Throttle · brake, reverse
A · D · Space	Steer · handbrake
Drone: Space · Shift	Climb · descend

KEY NUMBERS

NAME	VALUE	WHERE
`EYE` · `PLAYER_R` · `STEP_UP`	`1.66 · 0.34 · 0.62`	00-config, 06-collision
`WALK` · `RUN` · `SPRINT`	`3.05 · 5.6 · 10.4`	17-player
`sitDrop` sit · lie	`0.66 · 1.34`	17-player
`FOV_ACROSS` · min · max	`88 · 50 · 66`	03-scene
Shadow map · box · `KEY_D`	`3072 · 92 m · 110`	03-scene
Bias · normal bias	`-0.00005 · 0.012`	03-scene
Tone shoulder `K`	`0.76`	03-scene
Light pool · reach	`12 · 46 m`	16-lighting
`DayNight.DUR`	`414 s`	05-sky
Fog near · far (day)	`185 · 1460 m`	19-loop
Flat no-cast threshold `FLAT_T`	`12 mm`	01-helpers
Collider grid cell · pad	`12 m · 1.4 m`	06-collision
Door auto-open · prompt · release	`2.5 · 3.4 · 4.6 m`	17-player
`CAR_RIDE`	`0.96 m`	10-office

PORTING CHECKLIST

To start a new world from this engine:

01 **Copy the shell and the engine files:** `index.html` (markup, CSS, HUD), `00`–`06`, `11`, and `15`–`19`. Keep three.js r128, or test carefully before upgrading; the tone-mapping patch replaces a shader chunk by its exact text.

02 **Delete the content files** (`07`–`14j` except `11`) and remove their script tags.

03 **Cut or stub the content hooks** in the core files. These names are defined by motel content but read by the engine:

- `06-collision`: `POOL`, `PAVED` and `poolFloorY` in `groundY`; `TRAFFIC` for the traffic colliders.
- `16-lighting`: `POOL_LIGHTS`.
- `17-player`: `POOL` (wading) and `OFFICE` (the desk bell).
- `18-hud`: `ROOMS` (vacancy), `OFFICE`, `POOL`.
- `19-loop`: the pool's `waterMat`, `poolFloorCaustic` and `poolGlow`; `NEON`, `VAC_NEON`, `TVWIN_MAT`, `SLOW_GREEN`; `BASE` in the debug handle.
- `15-bake`: the rooftop clutter after the bake reads `lvlY`, `WALL_H1`, `NORTH` and `EAST`.
- The cars' constants and builder (`CARC`, `CAR_RIDE`, `CARS`, `driving`) live in `10-office.js`; move them to a file of their own before `11-driving.js`.

04 **Set the config:** `SPAWN`, `EYE`, `PLAYER_R`, your site constants, and `NEXT_SCENE_URL` if the world hands off to another page.

05 **Repaint the cycle:** `DayNight.P`, the night haze, the start time and `DUR`.

06 **Replace the land:** a new `heightAt` (base noise, features, pads), ring boxes on one lattice, and `SKIRT_GAPS` for anything you dig.

07 **Draw your textures** in `02-textures.js`, using the three-pass recipe, and declare their buckets in `15-bake.js`.

08 **Build content files in order**, one place per file, each in its own local frame, registering colliders, flats, zones, lamps, doors and seats as you go.

09 **Keep the light budget:** one key, twelve pool lamps, and glow for everything else.

10 **Keep the debug handle** (`window.MOTEL` or your own) and run the four checks on everything you add.

TWELVE SENTENCES TO REMEMBER

01 Paint the colour; let the light shade it; never let it clip.

02 An emissive does not illuminate.

03 A lamp with a volume is on only while you are inside it.

04 Do not raise the normal bias to cure acne; find what is shadowing itself.

05 Under the ground there is no sky.

06 You stand on the triangles that are drawn.

07 Every slab needs a flat, and flats overlap.

08 A hole goes through every course, including the ones you cannot see.

09 A lintel stops where it is.

10 A seat looks the other way from its chair.

11 Two faces on one plane fight; two faces that butt do not.

12 When a probe and a picture disagree, the probe is testing the wrong thing.

Low Desert Engine: a field guide. Written from the source of the Low Desert Motel (repository `a-better-internet/motel`), Round 48, October 2026. Screenshots are rendered by the game itself in a headless browser, with no retouching.